# TASK: WORDPRESS WEBSITE HOSTING WITH SFTP, PHPMYADMIN AND SSL

## Objective

To host a WordPress website on an AWS EC2 RHEL server with Nginx and PHP-FPM, create a MariaDB database, configure secure SFTP access, install and configure phpMyAdmin, enable HTTPS using a free SSL certificate, and customize the WordPress dashboard with a personal blog post.

## Requirements

- AWS EC2 instance with RHEL
- Nginx web server
- PHP 8+
- PHP-FPM
- MariaDB database server
- WordPress
- SFTP
- phpMyAdmin
- DuckDNS domain
- Let’s Encrypt SSL certificate

## Procedure

## Step 1 – Update the System

The system was first updated to make sure the installed packages were up to date.

sudo dnf update -y

## Step 2 – Install Nginx

Nginx was installed and enabled as the web server.

sudo dnf install nginx -y

Start and enable Nginx:

sudo systemctl enable --now nginx

Check the status:

sudo systemctl status nginx

## Step 3 – Install PHP and PHP-FPM

PHP and PHP-FPM were installed for running PHP-based applications such as WordPress and phpMyAdmin.

sudo dnf install php php-fpm php-mysqlnd php-gd php-mbstring php-xml php-json php-cli -y

Start and enable PHP-FPM:

sudo systemctl enable --now php-fpm

Check the PHP version:

php -v

Check PHP-FPM:

sudo systemctl status php-fpm

## Step 4 – Install MariaDB

MariaDB was installed as the MySQL-compatible database server.

sudo dnf install mariadb-server -y

Start and enable MariaDB:

sudo systemctl enable --now mariadb

Check the status:

sudo systemctl status mariadb

Check the installed database version:

mariadb --version

The installed version was MariaDB 10.11.18.

## Step 5 – Secure MariaDB

The MariaDB security configuration was performed using:

sudo mariadb-secure-installation

The following security options were configured:

- Root password was set.
- Anonymous users were removed.
- Remote root login was disabled.
- Test database was removed.
- Privilege tables were reloaded.

This improves the security of the database server.

```
((certbot-env) ) [ec2-user@ip-172-31-2-240 ~]$ mysql --version
mysql  Ver 15.1 Distrib 10.11.18-MariaDB, for Linux (x86_64) using  EditLine wrapper
((certbot-env) ) [ec2-user@ip-172-31-2-240 ~]$ php -v
PHP 8.4.24 (cli) (built: Jul 29 2026 05:29:50) (NTS gcc x86_64)
Copyright (c) The PHP Group
Built by Red Hat, Inc.
Zend Engine v4.4.24, Copyright (c) Zend Technologies
    with Zend OPcache v8.4.24, Copyright (c), by Zend Technologies
((certbot-env) ) [ec2-user@ip-172-31-2-240 ~]$ nginx -v
nginx version: nginx/1.26.3
```

## Step 6 – Create the Website User and Directory

A separate Linux user was created for the website and SFTP access.

sudo useradd -m zenhal

A password was then assigned to the user:

sudo passwd zenhal

The website directory was created under the user's home directory.

sudo mkdir -p /home/zenhal/blog/public

Ownership was assigned to the website user:

sudo chown -R zenhal:zenhal /home/zenhal/blog

Permissions were configured:

sudo chmod 755 /home/zenhal
sudo chmod 755 /home/zenhal/blog

The website root is:

/home/zenhal/blog/public

## Step 7 – Configure Nginx

A separate Nginx configuration file was created:

sudo nano /etc/nginx/conf.d/zenhal.conf

The configuration was:

```
server {
    server_name zenhal.duckdns.org;

    root /home/zenhal/blog/public;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    location ~ \.php$ {
        include fastcgi_params;
        fastcgi_pass unix:/run/php-fpm/www.sock;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    }

    location ~ /\.ht {
        deny all;
    }
}
```

The Nginx configuration was tested:

sudo nginx -t

Then Nginx was restarted:

sudo systemctl restart nginx

```
server {
    server_name zenhal.duckdns.org;

    root /home/zenhal/blog/public;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    location ~ \.php$ {
        include fastcgi_params;
        fastcgi_pass unix:/run/php-fpm/www.sock;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    }

    location ~ /\.ht {
        deny all;
    }

    listen 443 ssl; # managed by Certbot
    ssl_certificate /etc/letsencrypt/live/zenhal.duckdns.org/fullchain.pem; # managed by Certbot
    ssl_certificate_key /etc/letsencrypt/live/zenhal.duckdns.org/privkey.pem; # managed by Certbot
    include /etc/letsencrypt/options-ssl-nginx.conf; # managed by Certbot
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem; # managed by Certbot

}
server {
    if ($host = zenhal.duckdns.org) {
        return 301 https://$host$request_uri;
    } # managed by Certbot


    listen 80;
    server_name zenhal.duckdns.org;
    return 404; # managed by Certbot
```

## Step 8 – Configure the MariaDB Database

MariaDB was accessed:

sudo mariadb

The WordPress database was created:

CREATE DATABASE wordpress;

A database user was created:

CREATE USER 'zenhal'@'localhost' IDENTIFIED BY 'YOUR_DATABASE_PASSWORD';

Privileges were granted:

GRANT ALL PRIVILEGES ON wordpress.* TO 'zenhal'@'localhost';

Privileges were reloaded:

FLUSH PRIVILEGES;

Exit MariaDB:

EXIT;

The database used for WordPress is:

Database Name: wordpress
Database User: zenhal
Database Host: localhost

## Step 9 – Configure SELinux

Since SELinux was kept enabled, the website directory was given the appropriate SELinux context.

sudo semanage fcontext -a -t httpd_sys_content_t "/home/zenhal/blog/public(/.*)?"

The context was applied:

sudo restorecon -Rv /home/zenhal/blog/public

This allows Nginx to access the website files while keeping SELinux enabled.

## Step 10 – Download and Install WordPress

The website directory was opened:

cd /home/zenhal/blog

WordPress was downloaded:

sudo curl -O https://wordpress.org/latest.tar.gz

The archive was extracted:

sudo tar -xzf latest.tar.gz

WordPress files were copied into the website root:

sudo cp -a wordpress/. public/

The extracted directory and archive were removed:

sudo rm -rf wordpress latest.tar.gz

Ownership was configured:

sudo chown -R zenhal:zenhal /home/zenhal/blog/public

SELinux contexts were restored:

sudo restorecon -Rv /home/zenhal/blog/public

## Step 11 – Configure WordPress Database Connection

During the WordPress setup, the following database details were provided:

Database Name: wordpress
Username: zenhal
Password: YOUR_DATABASE_PASSWORD
Database Host: localhost

WordPress was then installed through the browser.

The WordPress administrator account was created with:

Username: zenhal
Password: YOUR_WORDPRESS_ADMIN_PASSWORD

## Step 12 – Configure SFTP

SSH password authentication was enabled for the SFTP user.

The configuration file was edited:

sudo nano /etc/ssh/sshd_config.d/50-cloud-init.conf

The following setting was configured:

PasswordAuthentication yes

The SSH configuration was tested:

sudo sshd -t

The effective setting was verified:

sudo sshd -T | grep -i passwordauthentication

The output confirmed:

passwordauthentication yes

SSH was restarted:

sudo systemctl restart sshd

## Step 13 – Configure SFTP Chroot

To restrict the zenhal user to its own website area, an SFTP chroot configuration was added to:

sudo nano /etc/ssh/sshd_config

The following configuration was added:

```
Match User zenhal
   ChrootDirectory /home/zenhal
   ForceCommand internal-sftp
   X11Forwarding no
   AllowTcpForwarding no
```

For the chroot to work correctly, /home/zenhal was owned by root:

```
sudo chown root:root /home/zenhal
sudo chmod 755 /home/zenhal
```

The SSH configuration was tested:

```
sudo sshd -t
```

The effective SFTP settings were verified:

```
sudo sshd -T -C user=zenhal,host=localhost,addr=127.0.0.1 | grep -E
'chrootdirectory|forcecommand|allowtcpforwarding|x11forwarding'
```

The SSH service was restarted:

```
sudo systemctl restart sshd
```

The SFTP user is now restricted to:

/home/zenhal

When connected through FileZilla, this directory appears as /.

The website can then be accessed through:

/blog/public

The user cannot browse the actual server directories such as /etc, /var, /usr, or /root.

## Step 14 – Test SFTP Connection

SFTP was tested from the Windows Command Prompt:

```
sftp zenhal@3.7.68.148
```

After successful login:

```
pwd
```

The SFTP root was displayed.

The website directory was accessed:

cd blog/public

Then:

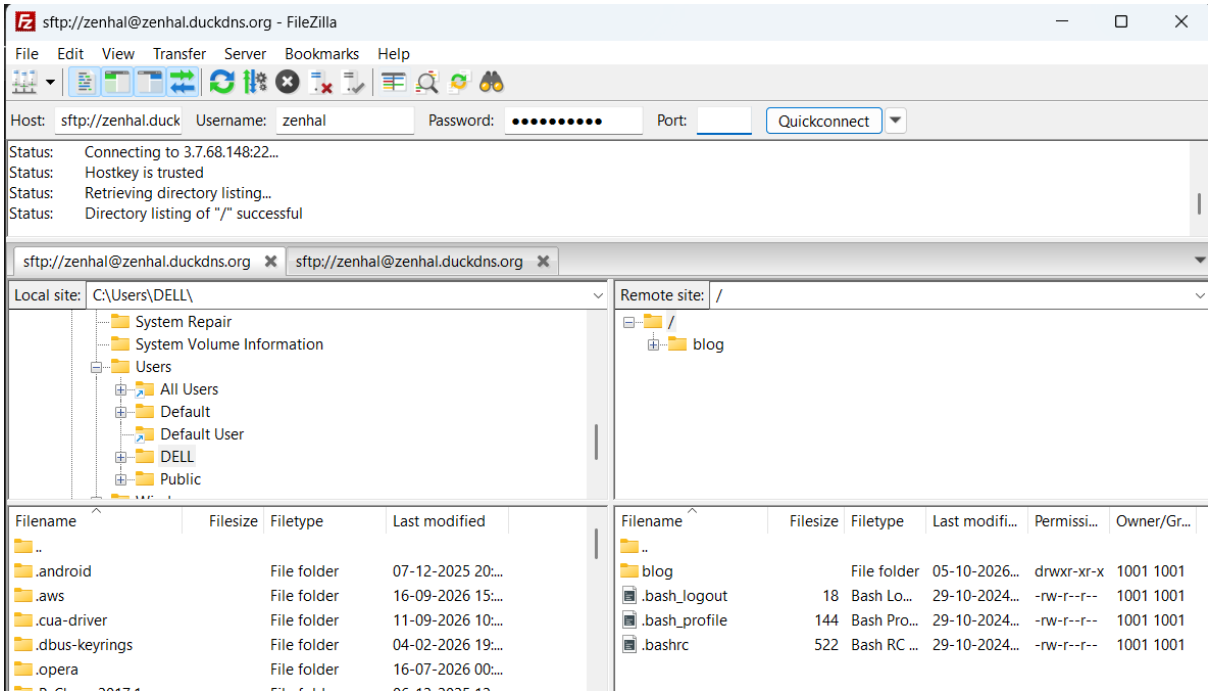
pwd

The website directory was displayed as:

/home/zenhal/blog/public

This confirmed that SFTP access was working correctly.

```
C:\Users\DELL\Downloads>sftp zenhal@3.7.68.148
zenhal@3.7.68.148's password:
Connected to 3.7.68.148.
sftp> pwd
Remote working directory: /home/zenhal
sftp> ls
blog
sftp> cd blog/public
sftp> pwd
Remote working directory: /home/zenhal/blog/public
sftp> exit
```



## Step 15 – Install phpMyAdmin

phpMyAdmin was manually installed because a suitable package was not available from the configured repositories.

The website directory was opened:

cd /home/zenhal/blog/public

phpMyAdmin was downloaded:

sudo curl -O https://files.phpmyadmin.net/phpMyAdmin/5.2.3/phpMyAdmin-5.2.3-all-languages.tar.gz

The archive was extracted:

sudo tar -xzf phpMyAdmin-5.2.3-all-languages.tar.gz

The extracted directory was renamed:

sudo mv phpMyAdmin-5.2.3-all-languages phpmyadmin

The archive was removed:

sudo rm phpMyAdmin-5.2.3-all-languages.tar.gz

Ownership was configured:

sudo chown -R zenhal:zenhal /home/zenhal/blog/public/phpmyadmin

SELinux context was restored:

sudo restorecon -Rv /home/zenhal/blog/public/phpmyadmin

## Step 16 – Configure phpMyAdmin

The phpMyAdmin sample configuration was copied:

cd /home/zenhal/blog/public/phpmyadmin
sudo cp config.sample.inc.php config.inc.php

The configuration file was edited:

sudo nano config.inc.php

The authentication method was configured as:

```
$cfg['Servers'][$i]['auth_type'] = 'cookie';
$cfg['Servers'][$i]['host'] = 'localhost';
$cfg['Servers'][$i]['AllowNoPassword'] = false;
```

A Blowfish secret was generated using:

```
openssl rand -base64 32
```

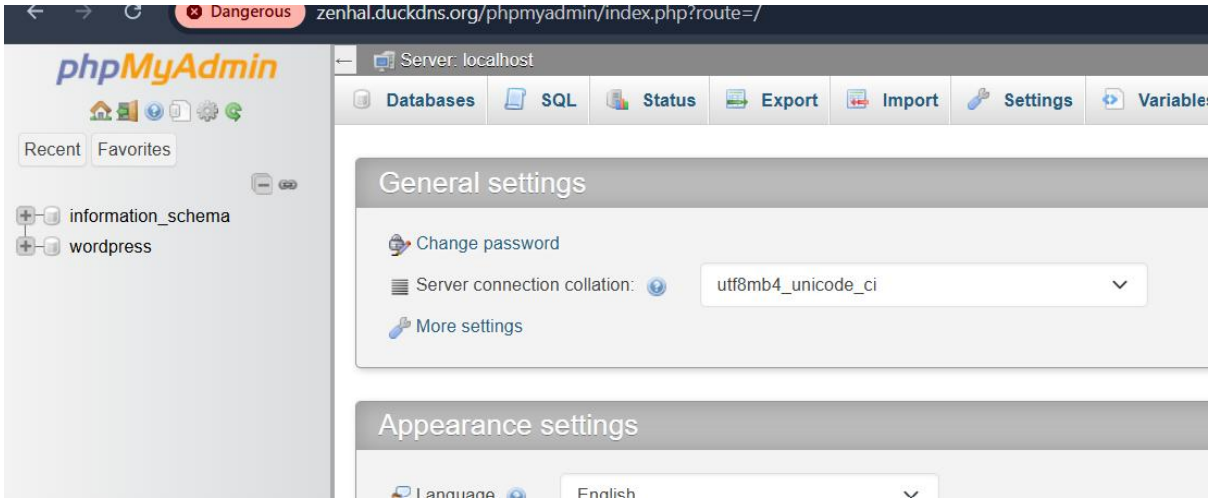
The generated value was added to the phpMyAdmin configuration.

The configuration syntax was checked:

```
sudo php -l /home/zenhal/blog/public/phpmyadmin/config.inc.php
```

phpMyAdmin was then accessed through:

https://zenhal.duckdns.org/phpmyadmin




## Step 17 – Configure DuckDNS Domain

The domain used for the website was:

zenhal.duckdns.org

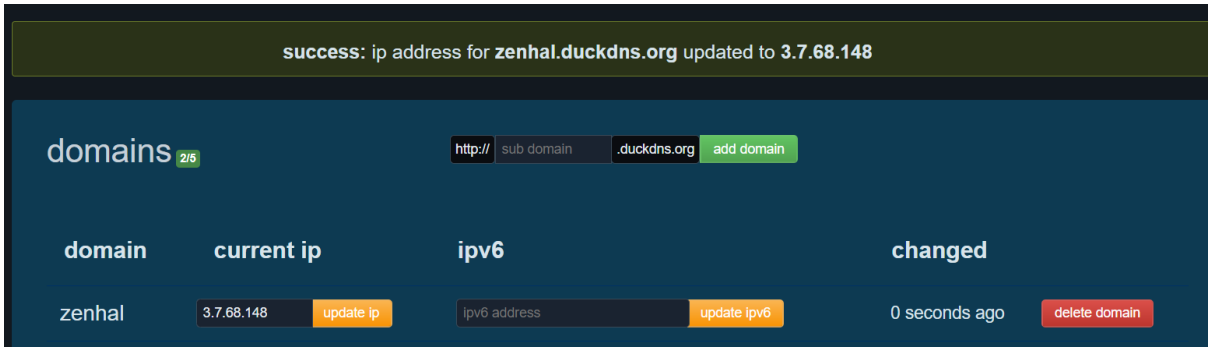
The domain was configured to point to the EC2 public IP address:

3.7.68.148

DNS resolution was verified using:

```
nslookup zenhal.duckdns.org
```

The domain resolved to the EC2 public IP address.




## Step 18 – Install SSL Certificate

A Python virtual environment was created for Certbot:

```
cd ~
python3 -m venv certbot-env
```

The environment was activated:

```
source ~/certbot-env/bin/activate
```

Certbot was installed:

```
pip install certbot
```

The Nginx plugin was installed:

```
pip install certbot-nginx
```

The installed Certbot version was checked:

```
certbot --version
```

Certbot plugins were verified:

```
certbot plugins
```

The SSL certificate was then requested for the domain:

```
sudo /home/ec2-user/certbot-env/bin/certbot --nginx -d zenhal.duckdns.org
```

Certbot successfully generated the certificate and configured HTTPS for Nginx.

The certificate files were stored under:

```
/etc/letsencrypt/live/zenhal.duckdns.org/
```

The certificate can be checked using:

sudo ls -l /etc/letsencrypt/live/zenhal.duckdns.org/

The website can now be accessed securely through:

https://zenhal.duckdns.org

```
Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/zenhal.duckdns.org/fullchain.pem
Key is saved at:         /etc/letsencrypt/live/zenhal.duckdns.org/privkey.pem
This certificate expires on 2027-01-03.
These files will be updated when the certificate renews.
```

## Step 19 – WordPress Dashboard Customization

The WordPress dashboard was customized with the following details:

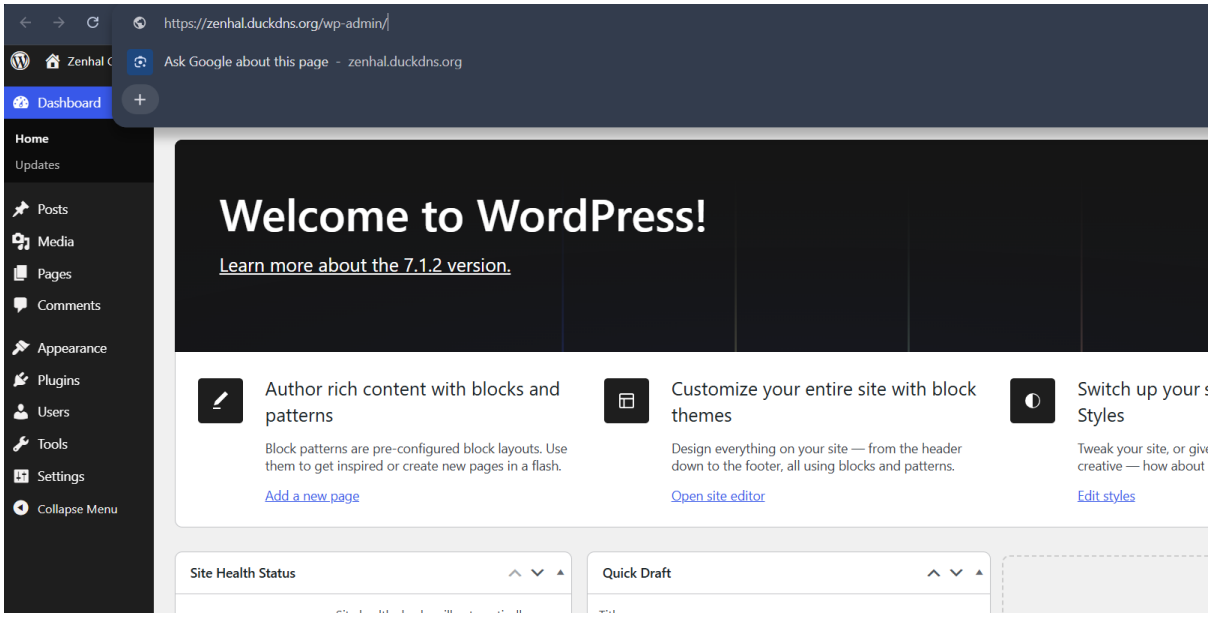
Site Title:
Zenhal Cloud & DevOps Blog

Tagline:
Cloud & DevOps Learning Journey

The Twenty Twenty-Five theme was used.

The WordPress administrator dashboard was accessed through:

https://zenhal.duckdns.org/wp-admin



## Step 20 – Create Personal Blog Post

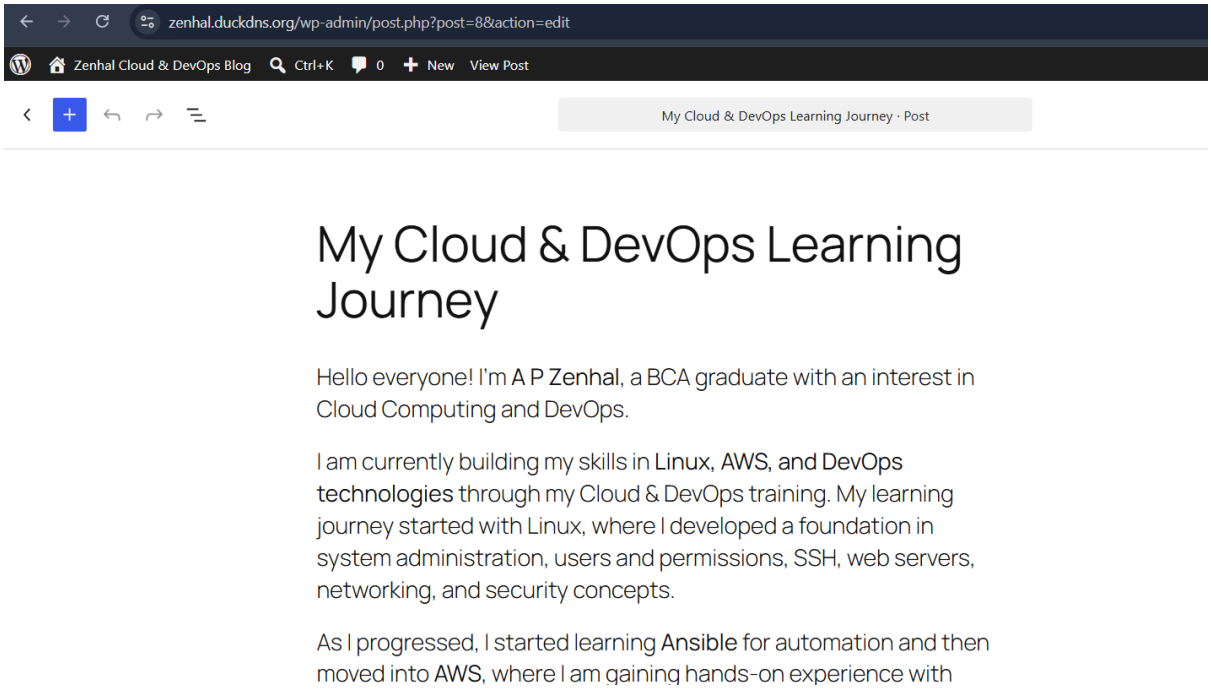
A personal blog post was created in WordPress.

Title:

My Cloud & DevOps Learning Journey

The post describes the learning journey from BCA to Cloud and DevOps, including Linux, Ansible and AWS learning.

The post was published from the WordPress dashboard.




## Verification

The complete setup was verified using the following URLs:

WordPress Website
https://zenhal.duckdns.org
WordPress Administrator
https://zenhal.duckdns.org/wp-admin
phpMyAdmin
https://zenhal.duckdns.org/phpmyadmin
SFTP
Host: 3.7.68.148

Username: zenhal
Port: 22
Protocol: SFTP

The following components were successfully verified:

- Nginx web server running
- PHP and PHP-FPM working
- MariaDB database running
- WordPress website accessible
- SFTP connection working
- SFTP user restricted using chroot
- phpMyAdmin accessible
- DuckDNS domain resolving correctly
- HTTPS working with Let's Encrypt
- WordPress dashboard accessible
- Personal blog post published

## Credentials

## SFTP

Host: 3.7.68.148
Username: zenhal
Password: [SFTP PASSWORD]
Port: 22
Protocol: SFTP

**phpMyAdmin**

URL: https://zenhal.duckdns.org/phpmyadmin
Username: zenhal
Password: [DATABASE PASSWORD]

**WordPress**

Admin URL: https://zenhal.duckdns.org/wp-admin
Username: zenhal
Password: [WORDPRESS ADMIN PASSWORD]

## Conclusion

The WordPress website was successfully hosted on an AWS EC2 RHEL server using Nginx and PHP-FPM. MariaDB was configured as the database server, and WordPress was connected to the database successfully.

Secure SFTP access was configured for website file management, with the user restricted using an SFTP chroot environment. phpMyAdmin was installed for database management.

The DuckDNS domain was configured and a free Let's Encrypt SSL certificate was installed, enabling secure HTTPS access.

Finally, the WordPress dashboard was customized and a personal blog post describing the Cloud & DevOps learning journey was published.